Phase 06 — Revision Sheet (Active Recall)

> *Do NOT open the notes. Answer aloud/on paper, then check and mark ✅/❌. Re-test ❌ items tomorrow. Review: after phase → +1 day → +3 days → +1 week → +1 month.*

A. Rapid-fire recall

1. What problem do containers solve? ("works on my machine")
2. Container vs VM — what is shared, what is isolated? Why are containers lighter?
3. Image vs container — the class/object analogy. Registry?
4. What are image layers? The Dockerfile ordering rule for cache efficiency?
5. Why multi-stage builds for Java? What stays OUT of the final image?
6. What does `-p 8080:80` mean exactly (which side is host)?
7. How do two containers talk to each other? What does Docker's internal DNS resolve?
8. Why does `localhost:5432` fail inside a container? What's the fix?
9. What happens to container data on removal? Named volume vs bind mount — use case of each?
10. Why does `docker compose down -v` deserve extra care?
11. In our compose stack, which service publishes ports and which are internal-only — and why is that the security model?
12. How does `depends_on` + healthcheck fix startup ordering?
13. Where do secrets go (and 2 places they must never go)?
14. Why is `:latest` in production a bug? What tag scheme instead?
15. Why not run as root inside containers?
16. The laptop→server shipping flow with a registry (build, tag, push, pull, run).

B. Write from memory

- The multi-stage Dockerfile for a Spring Boot app (both stages, key instructions).
- The compose stack diagram: nginx/api/db/redis — networks, ports, volumes marked.
- The JDBC URL when postgres runs as compose service `db`.

C. Command drill

```
docker ps -a              docker logs -f api         docker exec -it api sh
docker build -t app:1.0 . docker compose up -d       docker compose ps
docker compose stop api   docker volume ls           docker system prune
```

D. Self-score

- 13+ of A → move on. 8–12 → redo lab + re-test tomorrow. <8 → re-read notes actively, redo lab.